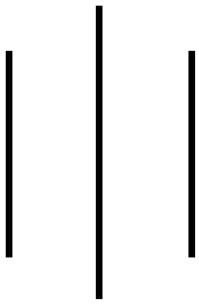
LA GRANDEE INTERNATIONAL COLLEGE

Simalchaur, Pokhara Nepal

A Final Report

On

"Parking Space Counter"



Submitted to:

Bachelor of Computer Application(BCA) Program

In partial fulfillment of the requirements for the degree of BCA under

Pokhara University

Submitted by:

Name:	Course	Semester	P.U. Registration Number
Saksham Giri	BCA	2nd	2025-1-53-0342
Kalyan Karki	BCA	2nd	2025-1-53-0324
Sandesh Wagle	BCA	2nd	2025-1-53-0346

Date: 12/07/2026

Acknowledgement

The satisfaction that accompanies after the successful completion of any task will be incomplete without mentioning the people whose ceaseless and relentless cooperation, constant guidance and encouragement made this project possible.

We are grateful to our project supervisor **Mr. Arpan Pokhrel**, our faculty teacher **Mr. Ramesh Chalise,** for the guidance, inspiration and constructive suggestions that helped us in the preparation of this project.

We are also appreciative among each other and have understood that teamwork, the designation of the task per the skillset one portrays, constant synchronisation and monitoring of progress and instilling new knowledge and skill is imperative for the success of any given work.

Sincerely,

Saksham Giri (25530436)

Sandesh Wagle (25530440)

Kalyan Karki (25530418)

Declaration for

"Parking Space Counter"

Student's Declaration

We hereby declare that we are the only authors of this work and that no sources other than the listed here have been used in this work.

_____

Saksham Giri (25530436)

Sandesh Wagle (25530440)

Kalyan Karki (25530418)

II Semester, BCA

Date: 12/07/2026

Supervisor's Declaration

I hereby recommend that this project entitled "**Parking Space Counter**" is done under my supervision by Saksham Giri, Sandesh Wagle and Kalyan Karki during their 2nd Semester in partial fulfillment of the requirements for the degree of **Bachelor of Computer Application** under **Pokhara University** is completed to my satisfaction and be processed for final evaluation.

Arpan Pokhrel

Supervisor

Date: 12/07/2026

Letter of Approval

We certify that we have examined this report entitled "**Parking Space Counter**" and are satisfied with the project defence. In our opinion it is satisfactory in the scope and qualify as project in partial fulfillment of the requirements for the degree of **Bachelor of Computer Application** under **Pokhara University.**

Er. Kiran KC	Er. Dikshan Gurung	Ankit Poudyal
Principal	Examiner	**Program Coordinator**

Date: 12/07/2026

Table of Contents

LIST OF FIGURES

List of Tables

ABBREVIATIONS

AD	Advertisement
CRUD	Create Read Update Delete
DOCS	Documents
IDE	Integrated Development Environment
MS	Microsoft
SDLC	Software Development Life Cycle

1. Introduction

The Parking Space Counter is a simple real-life project developed using the C programming language to manage and monitor parking availability in areas such as malls, schools, hospitals, and offices. The main purpose of this project is to count the number of vehicles entering and leaving a parking area and display the available parking spaces. This system helps reduce manual work, saves time, and prevents overcrowding in parking areas. It uses basic programming concepts such as variables, loops, conditions, and functions, making it suitable for beginners who want to learn practical applications of C programming while solving a common real-life problem.

This system helps to increase efficiency by organizing vehicle movement in a systematic way and reducing confusion in parking areas. Similarly, it also gives practical experience in developing management systems that can be applied in smart city and automation projects in the future. Since the project is simple and low-cost, it can be implemented easily in small parking areas without expensive technology. So, for that we developed the system called Parking Space Counter.

For our project, we divided work between us from the beginning. Documentation and coding was divided between all of us. Mainly, Gantt chart was sketched and requirement traceability matrix was done by Sandesh Wagle. Similarly, Waterfall Model was designed by Saksham Giri. Kalyan Karki designed Use Case Diagram. Research about project related information and testing platform and was handled by Arpan Pokhrel. However, each member contributed to the coding and testing of each module and provided their valuable inputs to improve the system.

While doing resea[i]rch we knew that there were many difficulties regarding collection of data, multiple seat booking, data was inaccurate, adaptation in system according to government policy was difficult while changing. By keeping these difficulties we made this system and we were successful to minimize these difficulties using C.

Going digitally is one of the great idea in this time period because each and every thing has been digitalized and many of the work pressures will be overcome, work will be much faster and we will be able to find each and every thing in much more efficient way. We will be able to deliver a better experience for our customers.Analyzing of data will be much more easier, so going digitally is best method.

2. Problem Statement

There were many difficulties as this was our first programming based project. Although there were many problems we were able to solve an implement it on our project. Manual parking management often causes confusion, errors in counting, and time delays. Drivers may struggle to find available space, and staff may fail to track vehicles properly. This project aims to solve these issues by automating the parking space counter process.

List of the following issues as secondary but prominent issues – can make problem statements in both micro and macro-level.

1. Lack of Efficient Parking Space Monitoring
2. Absence of Real-Time Information
3. Human Error in Manual System
4. Poor Utilization of Parking Space
5. Time Wastage and Traffic Delay

3. Objectives

The main objective of the Parking Space Counter project is to develop a simple system using the C programming language that can efficiently manage and monitor parking spaces. It aims to automatically track the number of vehicles entering and leaving a parking area and display the available and occupied spaces in real time. This helps to reduce the problems of manual counting, human errors, and confusion in parking management. The project also focuses on improving parking efficiency in places like malls, schools, hospitals, and offices, while saving time for users and ensuring proper use of available parking spaces.

1. To monitor vehicle entry and exit in the parking area.
2. To calculate available and occupied parking spaces.
3. To reduce errors caused by manual counting.
4. To provide real-time parking status information.
5. To improve efficiency in parking management systems.
6. To prevent overcrowding in parking areas.
7. To save time for drivers and parking staff.
8. To support better organization of parking facilities.

4. Background Study

The Parking Space Counter project is based on the study of real-life parking management problems faced in busy public and private places such as malls, hospitals, schools, offices, and markets. In these areas, the number of vehicles is increasing rapidly, but parking space is limited. Traditionally, parking is managed manually by attendants who count vehicles entering and exiting the area. This method is time-consuming, error-prone, and often leads to confusion about available parking spaces.

From this situation, it is observed that there is a need for a simple and efficient system to track parking space usage accurately. With the help of basic computer programming, especially C language, it is possible to design a system that can automatically count vehicles and display real-time parking status. The study of this problem shows how automation can improve accuracy, reduce human effort, and save time.

Therefore, this project is developed as a beginner-level software solution that applies fundamental programming concepts like variables, loops, conditions, and functions to solve a practical real-world problem related to parking management.

5. Requirement Document

A project cannot be successfully implemented without proper requirement analysis. For the Parking Management System, the requirements focus on managing vehicle parking records efficiently and accurately. The main purpose is to identify the system features needed to meet the administrator's requirements. This process defines the project scope, guides program development, and helps reduce errors during implementation.

The benefits include

Alignment: Defines the project scope and administrator requirements.

Direction: Provides guidance for program logic and file handling.

Efficiency & Productivity: Reduces coding errors and improves system reliability.

This document defines what the system must do, not how it is implemented. It also provides a basis for testing and validating the system.

5.1 Types of Requirements

1. Business Requirements: Provide a simple, reliable, and lightweight terminal-based system for managing vehicle parking records efficiently.

2. Users' Requirements: Securely log in, add and remove vehicles, view parking reports, modify parking rates, and save parking data.

3. Functional (Must Have) Requirements: User login, vehicle entry, vehicle exit, report generation, parking rate management, and save/load data.

4. Non-Functional Requirements: Local file storage (parking.txt), accurate parking fee calculations, input validation, and fast system response.

5.2 Requirements Matrix

SN.	Required modules, system, and features	Description for the modules	Priority	Remarks
1.	Login and Security System	Handles administrator login and authentication.	High	Validate username and password before granting access.
2.	Vehicle Management	Adds new vehicles and removes vehicles upon exit.	High	Ensure correct record entry and safe file updates.
3.	Parking Reports	Displays parked vehicles, parking fees, and summary reports.	Moderate	Reports should always reflect the latest saved data.
4.	Parking Rate Management	Allows the administrator to update parking charges.	Moderate	Validate new rates before saving changes.
5.	Save/Load Data	Stores and retrieves parking records using parking.txt.	High	Ensure reliable file operations to prevent data loss.

Table 5.1: Requirement Matrix

6. System Design

The Parking Space Counter is a system development in C language to monitor the number of vehicles entering and leaving a parking area. It automatically updates the available parking spaces and prevents vehicles from entering when the parking lot is full.

6.1 Data Flow Diagram

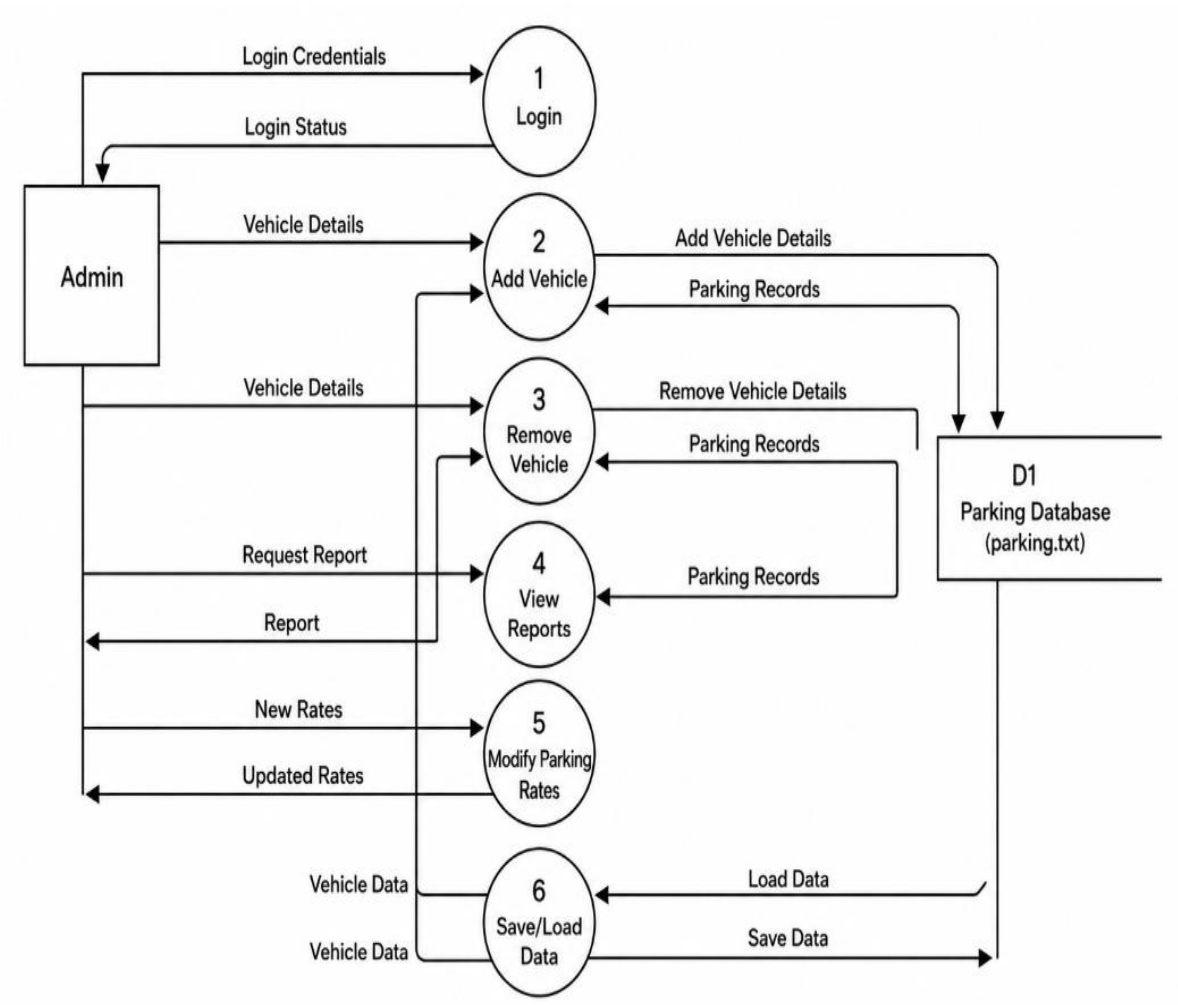


Figure 6.1: Level 1 DFD of Parking Space Counter

Description: This flowchart illustrates a parking management system process. It begins with a user request, validates vehicle data, checks for available spaces, updates the parking count, and finally displays the current status (Available/Full) along with parking records

6.2 System Flowchart

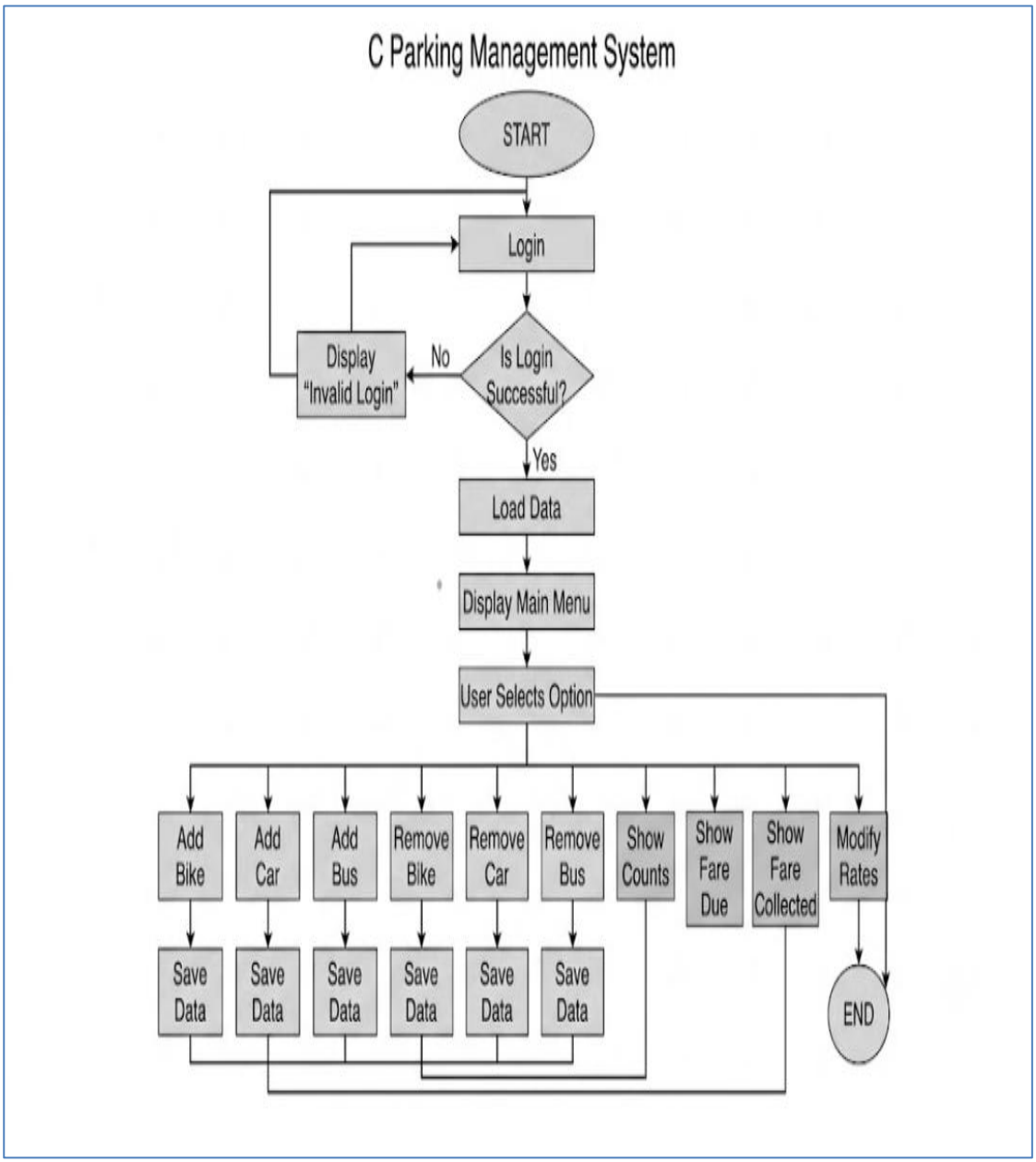


Figure 6.2: System Flowchart of Parking Space Counter

Description: This flowchart shows a simple parking system. It initializes capacity and count, then offers Entry, Exit, or View options. Entry adds a vehicle if space is available; Exit removes one; View shows available spots. The process loops until it ends.

6.3 Use Case Description

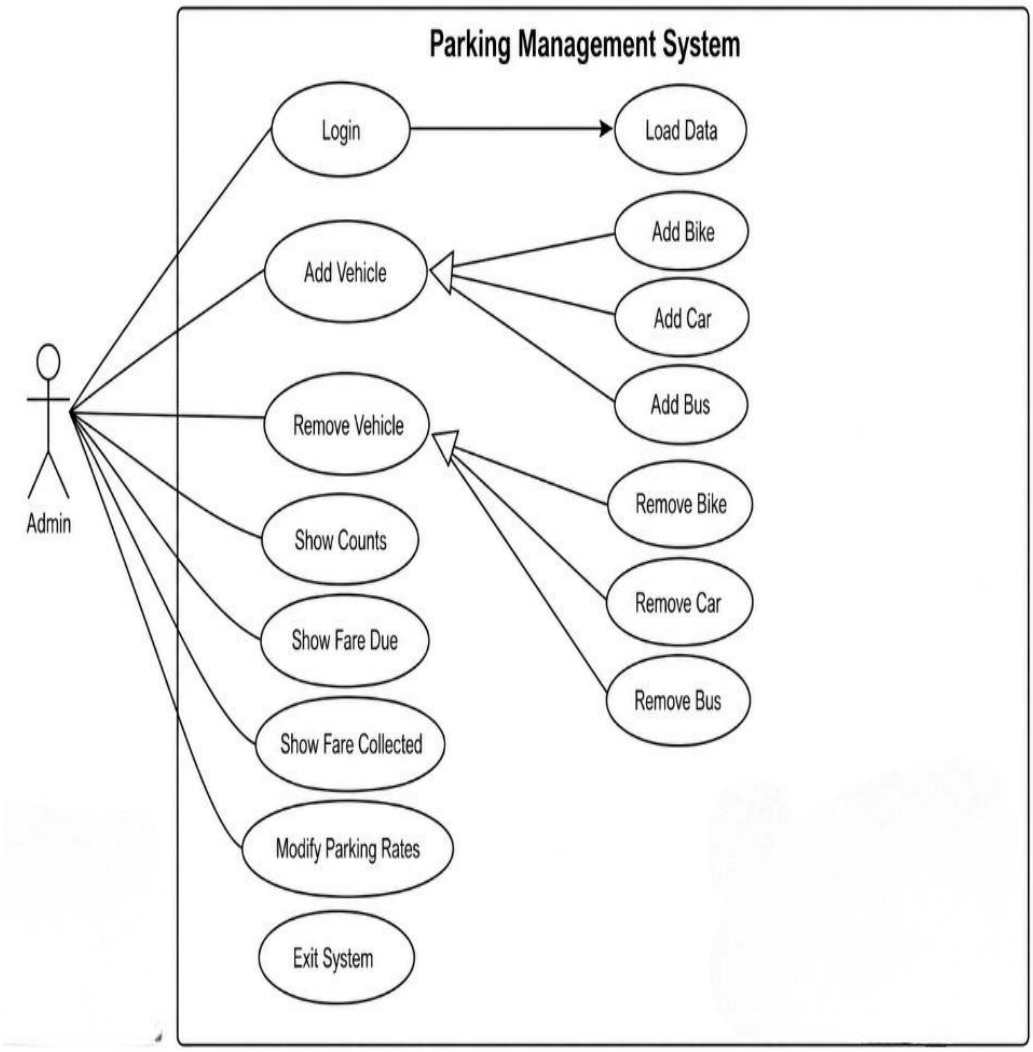


Figure 6.3: Use Case Diagram of Parking Space Counter

The Use Case diagram serves as a visual representation of the functional requirements of Parking Space Counter. It illustrates the interactions between the driver (The primary actor) and the various processes within the application.

6.4 Project Timeline (Gantt chart)

A Gantt chart is a project planning tools used to schedule and manage different activities within a specific time period. It helps in organizing tasks step by step and tracking the progress of the project.

For this project, the Gantt chart was used to divide the work into different phases such as planning, system design, coding, testing, and documentation. This helped in completing the project in a systematic and organized manner within the given time.

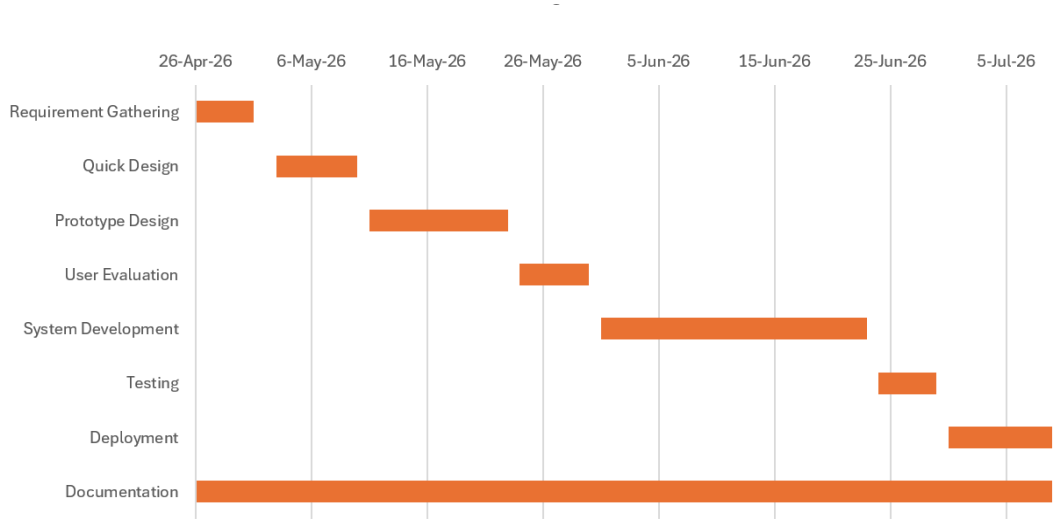


Figure 6.4: Gantt Chart of Parking Space Counter

7. Development

7.1 Software Development Life Cycle (SDLC) – Waterfall Model

For this project, the Waterfall Model has been used as the primary SDLC approach. The Waterfall Model is a linear and sequential development model in which each phase is completed one after another in a fixed order. It is suitable for this project because the requirements are clearly defined at the beginning and do not change frequently during development.

The phases include:

• **Requirement Gathering and Analysis:** Identifying the need for an automated parking system to manage vehicle entry, exit, and tracking available parking spaces.

• **System Design:** Designing the system architecture including processes such as vehicle entry, exit, counting logic, and status display using flowcharts and Data Flow Diagrams(DFDs).

• **Implementation:** Coding the system using the C programming language with concepts such as loops, conditions, functions, and variables to manage parking space calculations efficiently.

• **Testing:** Verifying the system to ensure correct handling of vehicle entry/exit operations and accurate display of parking availability and total count.

• **Deployment:** Running the final tested system in a working environment for real-time or simulated usage.

• **Maintenance:** Updating and fixing errors if any issues are found after deployment or during usage.

7.2 Development Environment and Tools

The following tools and environments were used in the development of the Parking Space Counter system:

- **Primary IDE & Toolchain (Dev-C++):** Used for writing, compiling, and debugging the C program. It provides a simple interface for testing program logic efficiently.
- **Programming Language (C):** Used to implement the core logic of the parking system including vehicle counting and space management.
- **Version Control System (GitHub):** Used for collaboration, code storage, and version tracking among team members, ensuring proper backup and teamwork coordination.
- **Documentation Tools (Pages):** Used for preparing and formatting the project report in a structured academic format.
- **System Design Tools (Draw.io / diagrams.net):** Used for creating flowcharts and Data Flow Diagrams (DFDs) to visually represent the system workflow and architecture.

8. Testing

Testing is the process of verifying and validating that a software application functions correctly and meets its specified requirements. The primary objective of testing is to identify errors, ensure system reliability, and confirm that all features operate as expected under different conditions.

For the Parking Space Counter system, testing was performed after the implementation phase to ensure that every module worked correctly. The login system, vehicle entry, vehicle exit, parking space counting, report generation, parking rate modification, and file handling were tested using different input scenarios. Both valid and invalid inputs were provided to verify the system's accuracy and stability.

Test Case ID	Menu Option	Test Scenario	Input	Expected Result	Status
TC-01	Login	Login with valid credentials	Username: admin Password: Hellothere123	Login successful and main menu is displayed	Pass
TC-02	Login	Login with invalid credentials	Incorrect username/password	"Invalid Username or Password" is displayed	Pass
TC-03	1. Add Bike	Add a bike	Valid bike number	Bike is added successfully and bike count increases	Pass
TC-04	2. Add Car	Add a car	Valid car number	Car is added successfully and car count increases	Pass
TC-05	3. Add Bus	Add a bus	Valid bus number	Bus is added successfully and bus count increases	Pass
TC-06	4. Remove Bike	Remove an existing bike	Existing bike number	Bike is removed and bike count decreases	Pass
TC-07	5. Remove Car	Remove an existing car	Existing car number	Car is removed and car count decreases	Pass

Test Case ID	Menu Option	Test Scenario	Input	Expected Result	Status
TC-08	6. Remove Bus	Remove an existing bus	Existing bus number	Bus is removed and bus count decreases	Pass
TC-09	7. Show Counts	Display vehicle counts	Select option 7	Current numbers of bikes, cars, buses, total parked vehicles, and available spaces are displayed correctly	Pass
TC-10	8. Fare Due	Calculate parking fee	Enter parked vehicle details	Correct parking fare due is displayed	Pass
TC-11	9. Fare Collected	View total collected fare	Select option 9	Total fare collected from all vehicles is displayed	Pass

Table 8.1: Test Summary

9. Project Results

The **Parking Management System** was developed to simplify vehicle parking operations, reduce manual record-keeping, and improve the accuracy of parking management. The objectives of the project were successfully achieved by providing a reliable and user-friendly terminal-based system.

The problems solved by the project are:

- **Accurate parking fee calculation:** The system automatically calculates parking **Reduced manual record management:** Administrators can quickly add and remove charges, reducing human calculation errors.
- **Organized data storage:** Parking records are securely stored in **parking.txt**, making them easy to access and manage.
- **Easy data backup:** Since records are stored in lightweight text files, they can be easily copied or backed up to prevent data loss.

After studying the limitations of manual parking management, we identified several challenges and addressed them through this digital system. The following needs were successfully met:

- **Improved parking management:** Vehicle entry, exit, and parking records can be managed efficiently from a single system.
- **Simple and user-friendly interface:** The menu-driven console interface allows administrators to perform operations easily without complex procedures.
- **Real-time parking information:** The system provides up-to-date parking records and reports whenever required.
- **Reduced data entry errors:** Input validation and secure login help minimize invalid records and unauthorized access.

- **Reliable file management:** Automatic saving and loading of records ensure data consistency and reduce the risk of information loss.

At the beginning of this project, we had limited experience in developing modular applications using the C programming language. Developing the Parking Management System was both challenging and rewarding, as it introduced us to concepts such as modular programming, file handling, user authentication, data validation, and report generation. Throughout the project, we improved our programming, problem-solving, and teamwork skills. This experience taught us that careful planning, continuous learning, and effective collaboration are essential for successfully completing a software development project.

10. Conclusion

The Parking Management System was successfully developed as a console-based application using the C programming language. The project provides an efficient way to manage parking operations by allowing the administrator to securely log in, add and remove vehicles, calculate parking fees, display parking statistics, modify parking rates, and store parking records in a file for future use.

The system minimizes the errors associated with manual parking management and improves the speed and accuracy of recording vehicle information. Through file handling, the application preserves parking data even after the program is closed, making it more practical and reliable than a purely memory-based system.

This project also enhanced our understanding of fundamental programming concepts such as functions, arrays, structures, file handling, modular programming, and user authentication. Overall, the Parking Management System demonstrates how a simple C-based application can effectively automate parking management tasks while providing a strong foundation for developing more advanced parking solutions in the future.

11. References

1. The C Programming Language. (1988). Prentice Hall.
2. Programming in ANSI C. (8th Edition). McGraw Hill Education.
3. Software Engineering. (10th Edition). Pearson Education.
4. GNU GCC Documentation
5. Microsoft Visual Studio Code Documentation

12. Annexures

Program output screenshots demonstrate the successful execution of the Parking Management System. They provide evidence that each module performs the intended functionality according to the project requirements.

Figure 12.1 Login Screen of the Parking Management System

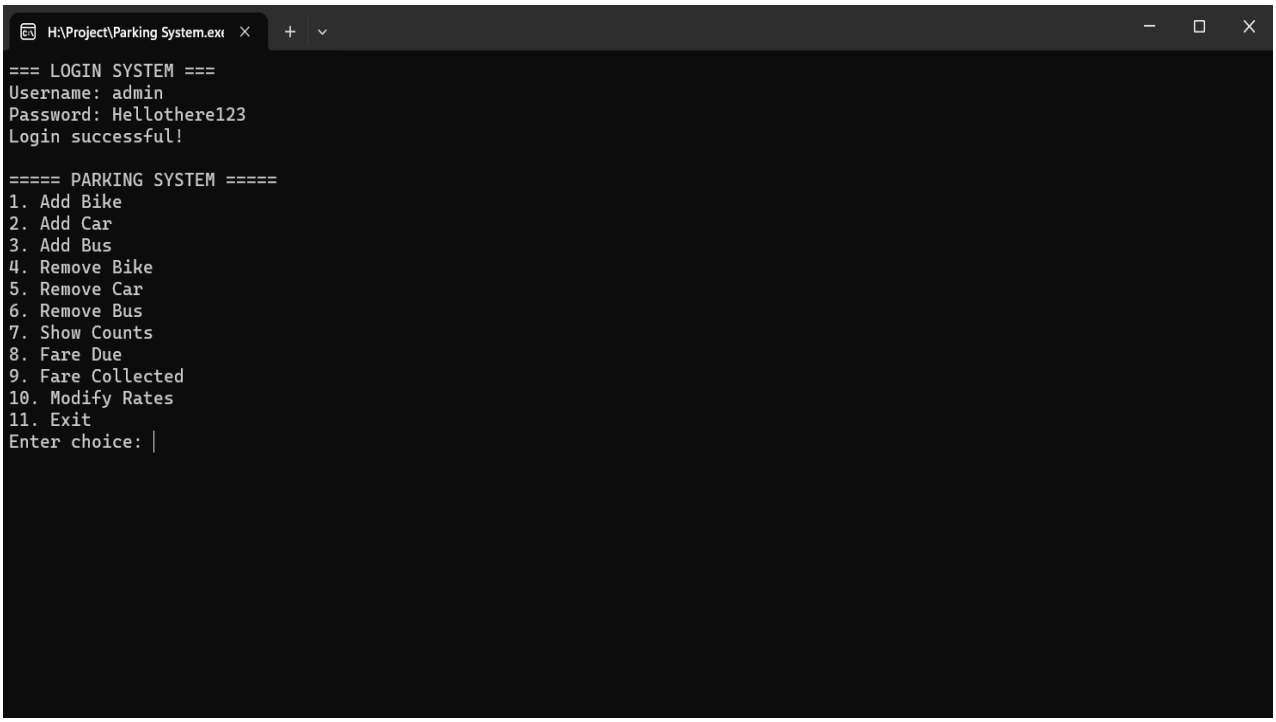


Figure 12.2 Main Menu Interface

```
===== PARKING SYSTEM =====
1. Add Bike
2. Add Car
3. Add Bus
4. Remove Bike
5. Remove Car
6. Remove Bus
7. Show Counts
8. Fare Due
9. Fare Collected
10. Modify Rates
11. Exit
Enter choice: 1
Enter bike number: 0000
Bike added successfully.
```

Figure 12.3Adding a Vehicle Record

```
===== PARKING SYSTEM =====
1. Add Bike
2. Add Car
3. Add Bus
4. Remove Bike
5. Remove Car
6. Remove Bus
7. Show Counts
8. Fare Due
9. Fare Collected
10. Modify Rates
11. Exit
Enter choice: 7

--- VEHICLE COUNT ---
Bikes: 1
Cars : 0
Buses: 0
Total: 1
```

Figure 12.4 Vehicle Count Report

```
===== PARKING SYSTEM =====
1. Add Bike
2. Add Car
3. Add Bus
4. Remove Bike
5. Remove Car
6. Remove Bus
7. Show Counts
8. Fare Due
9. Fare Collected
10. Modify Rates
11. Exit
Enter choice: 8

--- FARE DUE ---
Bikes: 1 x 20 = 20
Cars : 0 x 40 = 0
Buses: 0 x 60 = 0
TOTAL: 20
```

Figure 12.5 Fare Due Calculation

```
===== PARKING SYSTEM =====
1. Add Bike
2. Add Car
3. Add Bus
4. Remove Bike
5. Remove Car
6. Remove Bus
7. Show Counts
8. Fare Due
9. Fare Collected
10. Modify Rates
11. Exit
Enter choice: 9

--- COLLECTED (APPROX) ---
Bikes: 0 x 20 = 0
Cars : 0 x 40 = 0
Buses: 0 x 60 = 0
TOTAL: 0
```

Figure 12.7 Total Fare Collection Report

```
--- MODIFY RATES ---
1. Bike
2. Car
3. Bus
Choice: 1
New bike rate: 20

===== PARKING SYSTEM =====
1. Add Bike
2. Add Car
3. Add Bus
4. Remove Bike
5. Remove Car
6. Remove Bus
7. Show Counts
8. Fare Due
9. Fare Collected
10. Modify Rates
11. Exit
Enter choice:
```

Figure 12.6 Parking Rate Modification

```
Enter choice: 4
Bike removed. Collect 20 rupees.

===== PARKING SYSTEM =====
1. Add Bike
2. Add Car
3. Add Bus
4. Remove Bike
5. Remove Car
6. Remove Bus
7. Show Counts
8. Fare Due
9. Fare Collected
10. Modify Rates
11. Exit
Enter choice:
```

Figure 12.8 Remove Vehicle Operation

```
===== PARKING SYSTEM =====
1. Add Bike
2. Add Car
3. Add Bus
4. Remove Bike
5. Remove Car
6. Remove Bus
7. Show Counts
8. Fare Due
9. Fare Collected
10. Modify Rates
11. Exit
Enter choice: 11
Exiting...

--------------------------------
Process exited after 459.1 seconds with return value 0
Press any key to continue . . .
```

Figure 12.9 Program Exit Screen